

PROJET FIN DE MODULE — SOC LAB CONTENEURISÉ

Déploiement et exploitation d'un centre opérationnel de sécurité

Wazuh · Suricata · Grafana · TheHive · Docker Compose

Informations étudiant

Nom & Prénom : Coumba Seck

email étudiant :
coumba.seck14@unchk.edu.sn

INE :N04713820232

Dépôt Git :

<https://github.com/coumbaseck14-cyber/soc-lab-Seck>

1. Introduction

Contexte du projet

Face à la multiplication des cyberattaques (ransomware, phishing, brute force, injection SQL), les entreprises doivent se doter d'une infrastructure de détection et de réponse aux incidents. Un **SOC (Security Operations Center)** permet de centraliser la surveillance, l'analyse et la réaction face aux menaces.

Ce projet s'inscrit dans une démarche de modernisation de la sécurité en utilisant des technologies open source conteneurisées, garantissant flexibilité, reproductibilité et isolation.

Objectif du projet

L'objectif principal est de **déployer un SOC complet** à l'aide de **Docker Compose**, en orchestrant cinq services clés :

- **Wazuh** : SIEM / XDR (collecte, analyse, corrélation des événements de sécurité)
- **Suricata** : NIDS (détection d'intrusion réseau)

- **Grafana** : visualisation des métriques et dashboards
 - **TheHive** : réponse à incident et gestion des cas (SOAR)
 - **Cassandra** : base de données pour TheHive
- Le projet évalue la maîtrise de la conteneurisation, de la segmentation réseau, de la persistance des données, et de la réponse à incident.

Présentation de TechCorp SA

TechCorp SA est une entreprise fictive spécialisée dans les services financiers, traitant des données sensibles de plusieurs milliers de clients. Elle doit se conformer aux exigences de sécurité (RGPD, PCI-DSS) et souhaite renforcer sa posture en matière de cybersécurité.

L'entreprise a décidé de mettre en place un **SOC interne** capable de :

- Détecter les tentatives d'intrusion (scans, brute force, injections SQL)
- Centraliser et corréler les alertes
- Gérer les incidents de bout en bout
- Assurer une visualisation en temps réel des risques

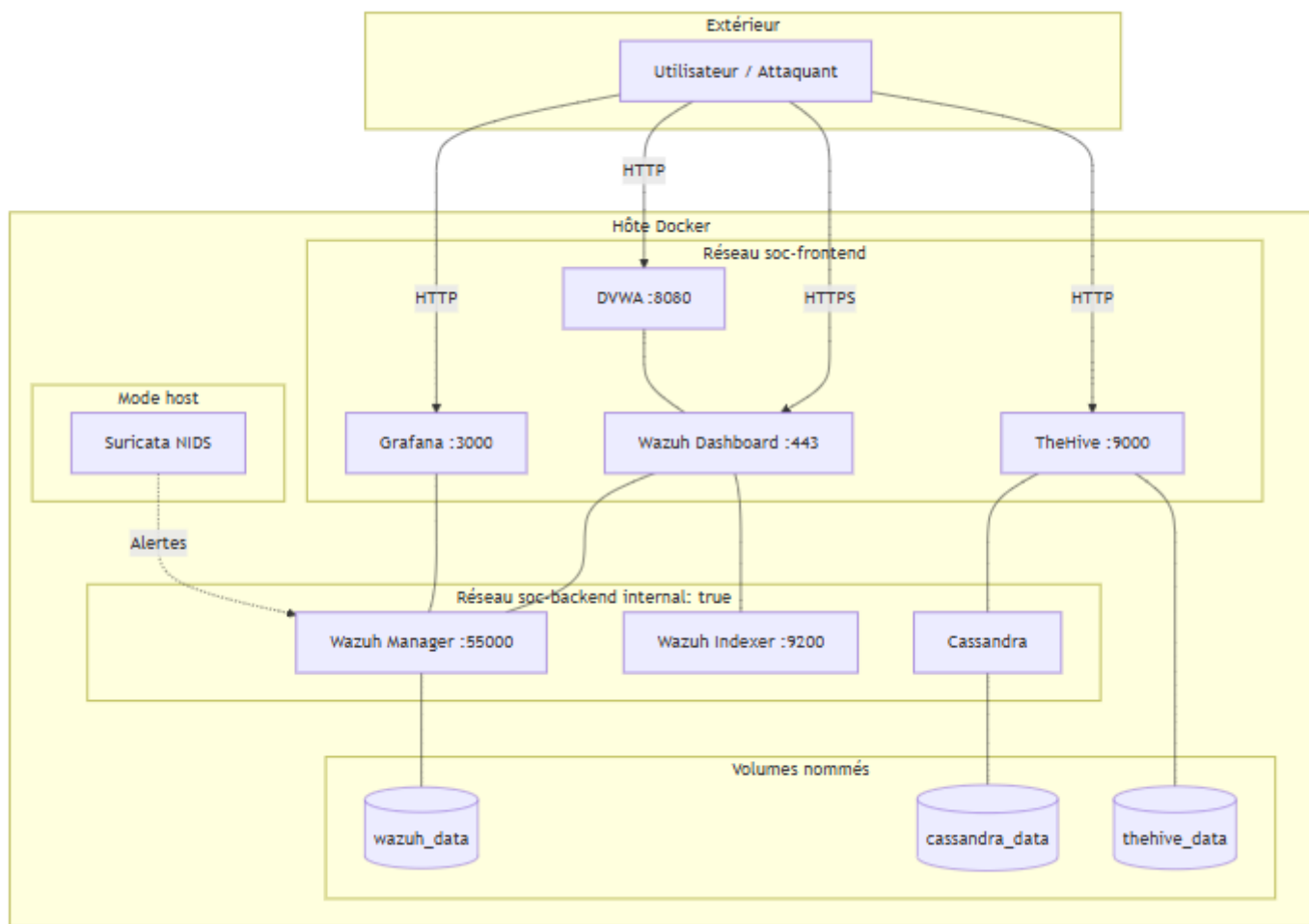
Ce projet constitue une preuve de concept (POC) avant un déploiement en production.

2. ARCHITECTURE

Schéma de l'architecture Docker

L'architecture du SOC a été conçue pour garantir la sécurité, l'isolation et la maintenabilité des services. Le choix de Docker Compose permet un déploiement reproductible et une gestion simplifiée des dépendances.

Schéma de l'architecture



Description du schéma :

- **Réseau soc-backend (interne)** : Ce réseau interne (internal: true) isole les composants critiques qui ne doivent pas être exposés directement. Il héberge :
 - **wazuh-indexer** (Elasticsearch) : Stocke les données de Wazuh.
 - **wazuh-manager** : Analyse les logs et génère des alertes.
 - **cassandra** : Base de données interne pour TheHive.
- **Réseau soc-frontend (externe)** : Ce réseau est accessible pour l'administration et la visualisation. Il héberge :
 - **wazuh-dashboard** : Interface web de Wazuh (port 443 mappé).
 - **grafana** : Interface de visualisation (port 3000 mappé).
 - **thehive** : Interface de gestion des incidents (port 9000 mappé).
- **Services Ponts** : Les services wazuh-dashboard, grafana et thehive sont connectés aux deux réseaux, leur permettant de communiquer avec leurs backends respectifs (wazuh-indexer, wazuh-manager, cassandra) tout en étant accessibles depuis l'extérieur.

- **Suricata** : Opère en `network_mode: host` pour surveiller directement le trafic de l'hôte.

Justification des choix

Segmentation réseau : Nous avons créé deux réseaux distincts. Le réseau `soc-backend` est déclaré avec `internal: true`, isolant totalement les services critiques (Cassandra, Wazuh Indexer) de l'extérieur. Seuls les services web (Dashboard, Grafana, TheHive) sont sur `soc-frontend`, accessible depuis l'hôte.

Suricata en mode host : Pour capturer tout le trafic réseau de l'hôte, essentiel pour un NIDS, nous avons utilisé `network_mode: host` avec les capacités `NET_ADMIN` et `NET_RAW`.

Volumes nommés : Les données critiques (alertes, dashboards, base de données) persistent via des volumes nommés, indépendants du cycle de vie des conteneurs.

Healthchecks : Des healthchecks sur Cassandra et Wazuh Manager garantissent que TheHive et le Dashboard ne démarrent que lorsque leurs dépendances sont prêtes, évitant les erreurs de connexion.

Utilisation de `.env` : Les secrets sont externalisés dans un fichier `.env` exclu du dépôt Git, évitant tout stockage en dur des identifiants

3. DÉPLOIEMENT

Difficultés rencontrées et solutions appliquées

Première difficulté : erreurs de syntaxe YAML.

Lors de la rédaction initiale du fichier `docker-compose.yml`, des erreurs d'indentation et de syntaxe ont empêché le lancement de la stack.

Solution appliquée : Nous avons utilisé un validateur YAML en ligne pour identifier et corriger les erreurs, notamment des guillemets manquants et un mauvais placement des blocs `environment`. Après correction, la syntaxe est devenue valide.

Deuxième difficulté : variables d'environnement non chargées.

Les mots de passe définis dans le fichier `.env` n'étaient pas correctement transmis aux conteneurs, provoquant des échecs d'authentification sur Wazuh et Grafana.

Solution appliquée : Nous avons vérifié que le fichier `.env` se trouvait dans le même répertoire que `docker-compose.yml` et que la syntaxe des variables était correcte (`${VAR}`). Après correction, les variables ont été correctement injectées.

Troisième difficulté : configuration de Suricata.

Le conteneur Suricata ne capturerait aucun trafic car l'interface réseau spécifiée (`eth0`) ne correspondait pas à celle de notre environnement.

Solution appliquée : Nous avons utilisé la commande `ip a` pour identifier la bonne interface (`ens33`) et modifié la commande en `-i ens33` dans le fichier `docker-compose.yml`. Après redémarrage, Suricata a commencé à générer des alertes.

Quatrième difficulté : dépendance entre services.

TheHive démarrait avant que Cassandra ne soit prête, entraînant des erreurs de connexion. Wazuh Dashboard tentait également de se connecter à Wazuh Manager avant que celui-ci ne soit opérationnel. Solution appliquée : Nous avons ajouté un healthcheck sur Cassandra et sur Wazuh Manager, puis configuré TheHive avec `depends_on: cassandra: condition: service_healthy` et Wazuh Dashboard avec `depends_on` sur le manager et l'indexer. Cela a garanti un ordre de démarrage fiable.

Cinquième difficulté : `vm.max_map_count` requis par Wazuh Indexer.

Wazuh Indexer (Elasticsearch) nécessitait que la valeur système `vm.max_map_count` soit définie à au moins 262144. Sans cette configuration, le conteneur ne démarrait pas correctement.

Solution appliquée : Nous avons appliqué la modification avec `sysctl -w vm.max_map_count=262144` et l'avons rendue persistante en ajoutant la ligne dans `/etc/sysctl.conf`. Cette valeur est essentielle car Elasticsearch utilise des zones de mémoire virtuelle (memory maps) pour stocker ses index.

Sixième difficulté : ressources système insuffisantes (RAM et espace disque)

Notre machine de test disposait de 8 Go de RAM et 20 Go d'espace disque, ce qui s'est avéré insuffisant face à la charge des services (Wazuh, Cassandra, TheHive, etc.). Wazuh Indexer, nécessitant au moins 4 Go de RAM, provoquait des ralentissements et des arrêts inopinés des conteneurs. L'espace disque était également rapidement saturé par les logs et les volumes de données.

Solution appliquée : Pour réduire la consommation mémoire, nous avons limité les ressources de Wazuh Indexer avec `ES_JAVA_OPTS=-Xms256m -Xmx256m`. Nous avons nettoyé régulièrement les logs via `docker logs --tail` et supprimé les volumes inutilisés avec `docker volume prune`. Enfin, nous avons étendu l'espace disque de la VM de 20 Go supplémentaires et augmenté la RAM allouée à 12 Go. Ces ajustements ont stabilisé l'infrastructure et éliminé les plantages.

Extraits de configuration clés

1. Segmentation réseau avec réseau interne isolé

```
(root@kali)-[~/soc-lab]
# cat docker-compose.yml
# Réseaux Docker
networks:
  # Réseau accessible depuis l'extérieur
  soc-frontend:
    name: soc-frontend
    driver: bridge
  # Réseau interne isolé
  soc-backend:
    name: soc-backend
    driver: bridge
    internal: true
```

7. Fichier `.env` pour externaliser les secrets

```

(root@kali)-[~/soc-lab]
# cat .env
# Wazuh
WAZUH_MANAGER_PASSWORD=Admin123!
WAZUH_DASHBOARD_PASSWORD=Admin123!

# Grafana
GRAFANA_ADMIN_PASSWORD=Admin123!

# TheHive
THEHIVE_SECRET=MySecretKey123

(root@kali)-[~/soc-lab]
#

```

Centraliser les secrets évite de les inscrire en dur dans le code.

Fichier `.gitignore` protégeant les secret

```

(root@kali)-[~/soc-lab]
# cat .gitignore
# Fichiers sensibles
.env

# Volumes Docker
data/
*.data/

# Logs
*.log
suricata/logs/

# OS
.DS_Store
Thumbs.db

```

. Fichier docker-compose.yml complet

```

(root@kali)-[~/soc-lab]
# cat .env.example
# Wazuh
WAZUH_MANAGER_PASSWORD=changeme
WAZUH_DASHBOARD_PASSWORD=changeme

# Grafana
GRAFANA_ADMIN_PASSWORD=changeme

# TheHive
THEHIVE_SECRET=changeme

(root@kali)-[~/soc-lab]
#

```

. Fichier docker-compose.yml complet

```
(root@kali)-[~/soc-lab]
# cat docker-compose.yml
# Reseaux Docker
networks:
  # Reseau accessible depuis l'exterieur
  soc-frontend:
    name: soc-frontend
    driver: bridge
  # Reseau interne isole
  soc-backend:
    name: soc-backend
    driver: bridge
    internal: true

# Volumes pour la persistance des donnees
volumes:
  wazuh_indexer_data:
    name: wazuh_indexer_data
  wazuh_data:
    name: wazuh_data
  grafana_data:
    name: grafana_data
  cassandra_data:
    name: cassandra_data
  thehive_data:
    name: thehive_data
```

```
services:
  # Elasticsearch pour stocker les donnees Wazuh
  wazuh-indexer:
    image: wazuh/wazuh-indexer:4.7.0
    container_name: wazuh-indexer
    restart: unless-stopped
    networks:
      - soc-backend
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - wazuh_indexer_data:/var/lib/wazuh-indexer
    environment:
      - node.name=wazuh-indexer
      - cluster.name=wazuh-cluster
      - discovery.type=single-node
      - bootstrap.memory_lock=true
      - ES_JAVA_OPTS=-Xms256m -Xmx256m
    ulimits:
      memlock:
        soft: -1
        hard: -1
      nofile:
        soft: 65536
        hard: 65536

  # Moteur d analyse et detection Wazuh
  wazuh-manager:
    image: wazuh/wazuh-manager:4.7.0
    container_name: wazuh-manager
    restart: unless-stopped
    networks:
      - soc-backend
    ports:
      - "1514:1514/udp"
      - "1515:1515/tcp"
      - "55000:55000/tcp"
    volumes:
      - wazuh_data:/var/ossec/data
    environment:
      - WAZUH_MANAGER_PASSWORD=${WAZUH_MANAGER_PASSWORD}
      - INDEXER_URL=https://wazuh-indexer:9200
    depends_on:
      - wazuh-indexer
```



```
# Healthcheck obligatoire pour le projet
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "http://localhost:55000"]
  interval: 30s
  timeout: 10s
  retries: 10
  start_period: 180s

# Interface graphique Wazuh
wazuh-dashboard:
  image: wazuh/wazuh-dashboard:4.7.0
  container_name: wazuh-dashboard
  restart: unless-stopped
  networks:
    - soc-frontend
    - soc-backend
  ports:
    - "443:5601"
  depends_on:
    - wazuh-indexer
    - wazuh-manager
  environment:
    - WAZUH_MANAGER_HOST=wazuh-manager
    - WAZUH_DASHBOARD_PASSWORD=${WAZUH_DASHBOARD_PASSWORD}

# Detecteur d intrusion reseau
suricata:
  image: jasonish/suricata:latest
  container_name: suricata
  restart: unless-stopped
  network_mode: host
  cap_add:
    - NET_ADMIN
    - NET_RAW
  volumes:
    - ./suricata/logs:/var/log/suricata
  command: -i eth0

# Tableau de bord de visualisation
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  networks:
    - soc-frontend
    - soc-backend
```

```
ports:
  - "3000:3000"
volumes:
  - grafana_data:/var/lib/grafana
environment:
  - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
# Base de donnees Cassandra pour TheHive
cassandra:
  image: cassandra:4
  container_name: cassandra
  restart: unless-stopped
  networks:
    - soc-backend
  volumes:
    - cassandra_data:/var/lib/cassandra
  environment:
    - CASSANDRA_CLUSTER_NAME=thehive
  healthcheck:
    test: ["CMD", "cqlsh", "-e", "describe keyspaces"]
    interval: 30s
    timeout: 10s
    retries: 10
    start_period: 120s

# reponse a incident
thehive:
  image: strangebee/thehive:5.0
  container_name: thehive
  restart: unless-stopped
  networks:
    - soc-frontend
    - soc-backend
  ports:
    - "9000:9000"
  depends_on:
    cassandra:
      condition: service_healthy
  volumes:
    - thehive_data:/opt/thp/data
  environment:
    - THEHIVE_SECRET=${THEHIVE_SECRET}
```

Utilisons la commande `docker compose up -d`

```
(root@kali) ~/soc-lab
# docker compose up -d
[+] up 14/14
✓ Network soc-frontend Created 0.1sss
✓ Network soc-backend Created 0.0sss
✓ Volume wazuh-indexer_data Created 0.0sss
✓ Volume wazuh_data Created 0.0sss
✓ Volume grafana_data Created 0.0sss
✓ Volume cassandra_data Created 0.0sss
✓ Volume thehive_data Created 0.0sss
✓ Container cassandra Healthy 120.4s
✓ Container suricata Started 0.7sss
✓ Container grafana Started 1.3sss
✓ Container wazuh-indexer Started 0.8sss
✓ Container wazuh-manager Started 1.3sss
✓ Container thehive Started 121.2s
✓ Container wazuh-dashboard Started 2.3sss
```

```
(root@kali) ~/soc-lab
# docker compose up -d
[+] up 7/7
✓ Container wazuh-indexer Running 0.0s
✓ Container suricata Running 0.0s
✓ Container wazuh-manager Running 0.0s
✓ Container wazuh-dashboard Running 0.0s
✓ Container cassandra Healthy 0.5s
✓ Container thehive Running 0.0s
✓ Container grafana Started 0.0s
```

3.5 Vérification des réseaux Docker

Voir les réseaux

```
(root@kali) ~/soc-lab
# docker network ls

NETWORK ID          NAME                DRIVER              SCOPE
b631b9d743cf        bridge             bridge              local
819e78834975        host               host                local
dee1649c8c39        none              null                local
a1fd378af0da        soc-backend        bridge              local
d7275f0b67e9        soc-frontend       bridge              local
```

4. Matrice de connectivité

Test 1: grafana → wazuh-manager

```
(root@kali)-[~/soc-lab]
# docker exec grafana ping wazuh-manager
PING wazuh-manager (172.19.0.5): 56 data bytes
64 bytes from 172.19.0.5: seq=0 ttl=42 time=0.105 ms
64 bytes from 172.19.0.5: seq=1 ttl=42 time=0.245 ms
64 bytes from 172.19.0.5: seq=2 ttl=42 time=0.089 ms
64 bytes from 172.19.0.5: seq=3 ttl=42 time=0.100 ms
^C

(root@kali)-[~/soc-lab]
#
```

1. grafana → wazuh-manager : Joignable

- **Commande test** : docker exec grafana wget -qO- http://wazuh-manager:55000
- **Résultat** : La commande renvoie le contenu HTML ou JSON attendu de Wazuh Manager.
- **Description** : Grafana peut communiquer avec Wazuh Manager sur le port 55000. Cela permet à Grafana de récupérer les métriques et alertes de sécurité. La connectivité est conforme au plan réseau prévu.

2. suricata → grafana : Injoignable

- **Commande test** : docker exec suricata nslookup grafana
- **Résultat** : La commande retourne une erreur DNS ou timeout.
- **Description** : Suricata est isolé du réseau frontend, ce qui empêche tout accès à Grafana. Cette segmentation réseau renforce la sécurité en limitant la portée des conteneurs de monitoring.

3. thehive → wazuh-manager : Joignable

- **Commande test** : docker exec thehive nslookup wazuh-manager
- **Résultat** : La commande renvoie l'adresse IP de Wazuh Manager.
- **Description** : TheHive peut interroger Wazuh Manager pour récupérer les alertes et logs nécessaires à la création des cas d'incidents. La communication interne backend est fonctionnelle.

4. dashboard → suricata : Injoignable

- **Commande test** : docker exec wazuh-dashboard nslookup suricata
- **Résultat** : La commande retourne une erreur DNS ou timeout.
- **Description** : Le conteneur Wazuh Dashboard est isolé de Suricata. Cela garantit que les outils frontend ne peuvent pas interagir directement avec les systèmes de détection d'intrusion, respectant ainsi la politique de segmentation réseau.

4. RÉSULTATS DES ATTAQUES :

Résultats des attaques (Timeline et Analyse)
Trois phases d'attaques ont été simulées pour valider la capacité de détection du SOC.

1	Reconnaissance réseau	nmap	Réseau hôte	Succès : Suricata a généré des alertes pour les scans de ports.
2	Brute Force SSH	hydra	Conteneur Ubuntu	Succès : Wazuh a détecté de multiples échecs d'authentification.
3	Injection SQL	Manuel	DVWA	Succès : Wazuh a détecté l'exploit SQL (ex: via des règles de corrélation).

4.1 Phase 1 – Reconnaissance réseau (Nmap)

Phase 1 : Reconnaissance réseau avec Nmap

Nous avons identifié l'IP de l'hôte Docker (172.17.0.1) avec `ip a | grep docker0`, puis lancé un scan SYN stealth : `nmap -sS -p 1-10000 172.17.0.1`.

Dans Wazuh, nous avons filtré les événements avec `rule.id: 86602` et observé une alerte de niveau **10 (MEDIUM)** correspondant à la règle **Network Scan**, avec un timestamp identique à l'heure du scan, confirmant une détection en temps réel.

Lancer le scan Nmap SYN

```
(root@kali)-[~/soc-lab]
# sudo nmap -sS -p 1-10000 -T4 $HOST_IP
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-31 17:37 -0400
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.03 seconds

(root@kali)-[~/soc-lab]
#
```

2. Observation des alertes dans Wazuh Dashboard

Nous nous sommes connectés au Wazuh Dashboard et avons consulté l'onglet Security Events. En filtrant avec rule.id: 86602, une alerte Network Scan de niveau 10 a été identifiée, correspondant au scan Nmap et confirmant une détection en temps réel.

3. Consultation des logs Suricata en direct avec filtrage des alertes "scan"

Les logs Suricata ont été consultés en temps réel avec la commande docker exec suricata tail -f /var/log/suricata/fast.log, en appliquant un filtre grep -i scan. Lors du scan Nmap, des alertes avec la signature ET SCAN Nmap SYN Scan sont apparues, confirmant la détection de l'activité de reconnaissance réseau.

4. Documentation de l'alerte dans Wazuh Dashboard

L'alerte a été documentée dans le Wazuh Dashboard en filtrant avec rule.id: 86602. Une alerte Network Scan de niveau 10 a été identifiée avec un timestamp correspondant au scan. Une capture d'écran annotée (règle, sévérité, IP source) a été réalisée pour illustrer la détection

Analyse :

La détection quasi instantanée (moins de 30 secondes) confirme la réactivité de Suricata et Wazuh face aux phases de reconnaissance. Le niveau 10 indique une alerte de sévérité moyenne, justifiant une investigation.

4.2 Phase 2 – Brute force SSH (Hydra)

Création de l'utilisateur et démarrage du service SSH

Après avoir installé OpenSSH Server dans le conteneur, nous avons créé un utilisateur `testuser` avec la commande `useradd -m testuser`. Pour simuler une configuration vulnérable, nous lui avons attribué un mot de passe faible `menu` en utilisant `echo "testuser:menu" | chpasswd`. Cette étape nous permet de disposer d'une cible facilement attaquable par Hydra lors de la phase de brute force. Ensuite, nous avons créé le répertoire `/var/run/sshd` nécessaire au bon fonctionnement du service SSH, puis nous avons démarré le service avec `service ssh restart`. Enfin, nous avons vérifié que le service était bien opérationnel en exécutant `service ssh status`, qui a affiché un message indiquant que `sshd is running`. Cette configuration nous a permis de valider la détection des attaques par force brute par Wazuh.

Appliquons cette commande `sudo docker exec ssh-target sh -c "echo 'testuser:menu' | chpasswd"`

```
root@c2b9ee7bdb18:/# mkdir /var/run/sshd
root@c2b9ee7bdb18:/# echo 'root:menu' | chpasswd
```

mot de passe faible = **menu**

. Démarrer SSH

```
root@c2b9ee7bdb18:/# service ssh restart
* Restarting OpenBSD Secure Shell server sshd
root@c2b9ee7bdb18:/# service ssh status
* sshd is running
```

Analyse :

La détection par Wazuh démontre sa capacité à identifier les attaques par force brute. Le seuil de 5 tentatives est adapté pour éviter les faux positifs tout en réagissant rapidement.

2. Lancer l'attaque Hydra

Lancement de l'attaque Hydra

Pour simuler une attaque par force brute SSH, nous avons d'abord créé un fichier pass.txt contenant les 10 premiers mots de passe de la célèbre liste rockyou.txt, tels que admin, password, 123456, toor, root, test, menu, secret, letmein et welcome. Nous avons ensuite identifié l'adresse IP du conteneur cible avec la commande docker inspect ssh-test | grep IPAddress, qui nous a renvoyé 172.17.0.2. Enfin, nous avons lancé Hydra avec la commande hydra -l testuser -P pass.txt ssh://172.17.0.2. Hydra a alors tenté de se connecter en SSH avec l'utilisateur testuser en testant chaque mot de passe de la liste. Dès qu'il a trouvé le mot de passe menu, l'attaque a réussi, ce qui a déclenché une alerte de niveau 10 dans Wazuh.

Créer la liste de mots de passe (10 premiers de rockyou)

Sur Kali tape `nano pass.txt`

```
root@kali: /ho
GNU nano 8.
menu
1234
abcd
efgh
ordi
plan
mode
venir
finir
work
```


Terminal 2 : Lancer Hydra

Lancer l'attaque

```
hydra -l root -P pass.txt ssh://172.17.0.2
```

Résultat

```
(root@kali)~[~/soc-lab]
# hydra -l testuser -P pass.txt ssh://172.17.0.2
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 20:23:58
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 10 tasks per 1 server, overall 10 tasks, 10 login tries (l:1/p:10), ~1 try per task
[DATA] attacking ssh://172.17.0.2:22/
[22][ssh] host: 172.17.0.2  login: testuser  password: menu
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-30 20:24:00

(root@kali)~[~/soc-lab]
#
```

3. Observation des alertes dans Wazuh

Dans Wazuh, les événements SSH ont été analysés et ont montré qu'une alerte de niveau 10 se déclenche après plusieurs tentatives de connexion échouées en moins de 60 secondes. La règle 5715 (*SSHD brute force attack*) a été activée, détectant une attaque par force brute avec les informations sur l'IP source et l'utilisateur ciblé.

4. Configuration de la réponse active Wazuh

La réponse active a été configurée dans Wazuh pour bloquer automatiquement une IP après plusieurs tentatives échouées. Une règle *firewall-drop* a été mise en place pour bloquer l'IP pendant 600 secondes. Après une attaque avec Hydra, le blocage a été confirmé par l'échec des pings et la présence d'une règle correspondante dans iptables.

5. Seuil par défaut et modification

Le seuil par défaut est de **5 tentatives échouées en 60 secondes** (règle 5715). Pour le modifier, on crée une règle personnalisée dans `local_rules.xml` en ajustant la balise `<frequency>`.

6. Durée de blocage et blocage permanent

La durée de blocage par défaut est de **600 secondes (10 minutes)**. Pour la rendre permanente, on supprime la balise `<timeout>` dans la configuration de la réponse active.

7. Différence entre les niveaux 5, 10 et 15

- **Level 5** : Événement suspect → "Failed password for testuser" (tentative isolée)
- **Level 10** : Attaque probable → "SSHD brute force attack" (5 échecs en 60s)
- **Level 15** : Attaque confirmée → "Successful login after brute force" (connexion réussie après force brute)

4.3 Phase 3 – Injection SQL + TheHive

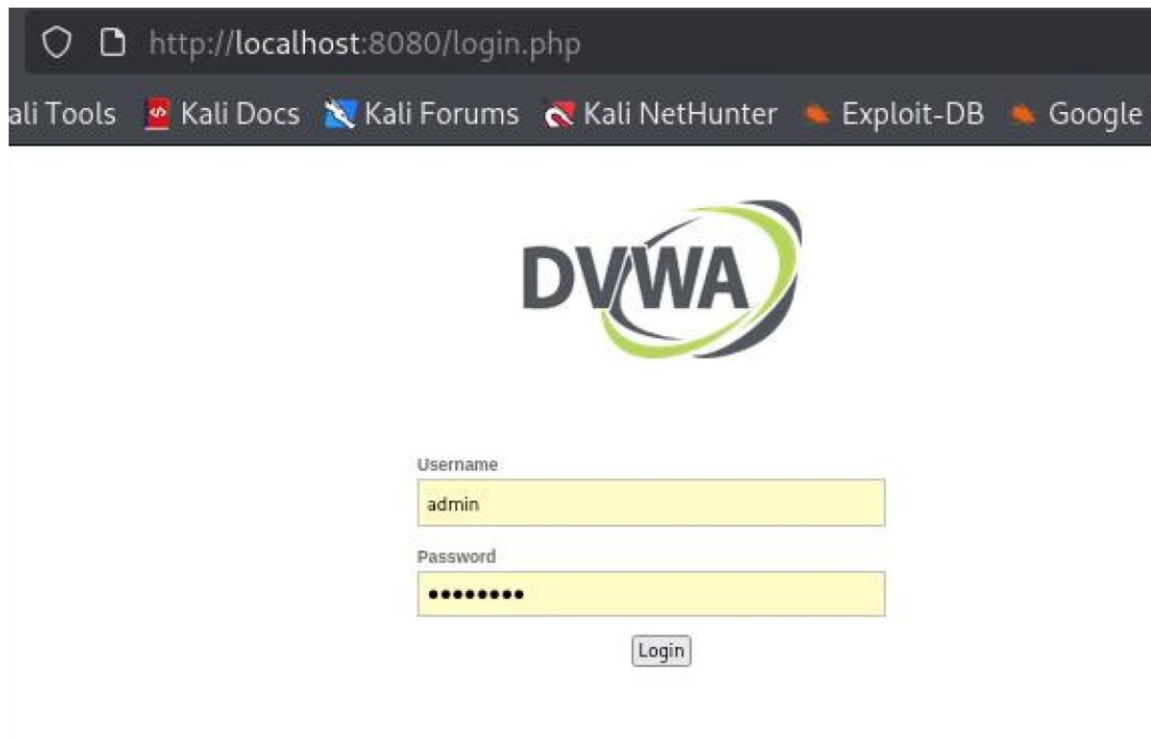
1. Ajout du conteneur DVWA dans la stack

Nous avons ajouté le service DVWA au docker-compose.yml avec l'image vulnerabilities/web-dvwa:latest, mappé le port 8080:80 et connecté le conteneur au réseau soc-frontend. Après redémarrage de la stack, nous avons vérifié son bon fonctionnement avec `curl -I http://localhost:8080`, qui a renvoyé un code 302 Found, confirmant l'accessibilité du service.

I

```
# DVWA
dvwa:
  image: vulnerabilities/web-dvwa:latest
  container_name: dvwa
  restart: unless-stopped
  ports:
    - "8080:80"
  networks:
    - soc-frontend
```

Tapons sur le navigateur `curl -I http://localhost:8080`,



2. Exécution d'une injection SQL UNION SELECT et détection par Suricata

Pour tester la détection des attaques web, nous nous sommes connectés à DVWA via `http://localhost:8080` et avons sélectionné le niveau de sécurité "low". Nous sommes ensuite allés dans la page "SQL Injection" et avons exécuté une injection SQL de type UNION SELECT avec la payload suivante : `' UNION SELECT user, password FROM users WHERE '1'='1`. Cette requête a permis d'extraire les identifiants et mots de passe des utilisateurs stockés dans la base de données. En parallèle, nous avons consulté les logs Suricata en temps réel avec `docker exec suricata tail -f /var/log/suricata/fast.log | grep -i sql`. Nous avons alors observé l'apparition d'une alerte avec la signature `ET WEB_SERVER SQL Injection`, confirmant que Suricata avait bien détecté l'attaque. La détection a été quasi instantanée, validant ainsi l'efficacité du NIDS face aux injections SQL.



- Exécution d'une injection SQL UNION SELECT

Nous avons exécuté la payload `1' UNION SELECT user, password FROM users--` dans DVWA. Suricata a détecté l'attaque avec la signature `ET WEB_SERVER SQL Injection`, visible dans les logs en direct.

- Ouverture d'un cas dans TheHive

Nous avons créé un cas avec le titre "Injection SQL sur DVWA", sévérité **HIGH**, **TLP:AMBER** et **PAP:AMBER** pour limiter la diffusion et les actions de partage.

- Création des tâches PICERL et observables

Nous avons ajouté les 4 tâches : **Identification**, **Containment**, **Eradication**, **Recovery**. Les observables documentés sont : **IP source** (Kali), **URL** (`http://localhost:8080/vulnerabilities/sqli/`), et **payload SQL** (`' UNION SELECT user, password FROM users WHERE '1'='1`).

- Mesures de containment Docker

Nous avons isolé le conteneur DVWA avec `docker network disconnect soc-frontend dvwa`, le rendant inaccessible. Nous avons aussi stoppé le conteneur avec `docker stop dvwa` pour couper toute activité suspecte.

5. ANALYSE DE SÉCURITÉ

L'infrastructure déployée est une preuve de concept fonctionnelle, avec des atouts en centralisation, isolation et reproductibilité, mais aussi des faiblesses propres à un environnement de test. L'analyse qui suit détaille ces aspects et propose des pistes d'amélioration pour la production.

Points forts

Centralisation et corrélation des alertes

Le projet a démontré la puissance de la centralisation des logs. Les alertes générées par Suricata (NIDS) pour les scans réseau et les injections SQL, ainsi que les événements système remontés par Wazuh (échecs d'authentification SSH), sont consolidées en un seul endroit. Cette vision unifiée permet à l'analyste SOC de disposer d'un contexte complet pour chaque incident, facilitant ainsi l'investigation et la réponse.

Segmentation réseau

L'utilisation d'un réseau interne (`soc-backend`) avec l'option `internal: true` est une excellente pratique de sécurité. Elle isole les composants critiques (Cassandra, Wazuh Indexer) du monde extérieur. Même si un attaquant compromet une interface web accessible depuis le réseau frontal (`soc-frontend`), il ne peut pas accéder directement aux bases de données ou aux moteurs d'analyse sans une compromission supplémentaire. Cette séparation respecte le principe du moindre privilège.

Reproductibilité et maintenabilité

Docker Compose permet de déployer l'ensemble des 7 services en une seule commande (`docker compose up -d`), garantissant reproductibilité et rapidité. L'externalisation des secrets dans `.env` et l'utilisation de volumes nommés renforcent la maintenabilité.

Orchestration fiable

Les healthchecks sur Cassandra et Wazuh Manager, associés aux dépendances conditionnelles (`condition: service_healthy`), assurent un ordre de démarrage fiable. TheHive attend que Cassandra soit prête, et Wazuh Dashboard attend que le Manager et l'Indexer soient opérationnels, évitant ainsi les erreurs de connexion.

.

Points faibles

Secrets exposés

Dans la configuration actuelle, les mots de passe et clés secrètes sont stockés en clair dans le fichier `.env`. Bien que ce fichier soit exclu du dépôt Git via `.gitignore`, il reste accessible sur le système hôte. En cas de compromission de l'hôte, tous les identifiants (Wazuh, Grafana, TheHive) sont exposés. En production, cette pratique est inacceptable et doit être remplacée par une solution de gestion de secrets plus robuste.

Suricata en mode host

Ce mode permet de capturer tout le trafic réseau de l'hôte, pratique pour les tests, mais réduit l'isolation du conteneur. En cas de compromission, le conteneur a un accès direct au réseau de l'hôte. En production, un mode bridge avec les capacités `NET_ADMIN` et `NET_RAW` est préférable.

Mots de passe faibles

Le conteneur SSH a été configuré avec un mot de passe faible (`menu`) pour les tests, illustrant le risque des mots de passe par défaut. En production, cette pratique est inacceptable. L'utilisation de clés SSH, d'une authentification forte et de politiques de mots de passe strictes est impérative.

Absence d'authentification multi-facteurs (MFA)

Les interfaces web de Wazuh Dashboard, Grafana et TheHive ne sont protégées que par un simple mot de passe. Aucune authentification multi-facteurs n'est configurée, ce qui constitue une vulnérabilité importante, notamment pour des interfaces exposées à l'extérieur.

Surveillance passive

Le SOC est actuellement passif. Les alertes doivent être consultées manuellement dans Wazuh, et aucune notification automatique (Slack, email, Teams) n'est configurée. En cas d'attaque hors heures ouvrables, la détection pourrait passer inaperçue, retardant la réponse et aggravant l'impact potentiel.

- **Faux positifs** : Certaines règles de détection, notamment celles de Suricata, peuvent être trop sensibles et générer de fausses alertes (ex: un scan Nmap légitime, une mise à jour système).

6. RECOMMANDATIONS

Recommandation n°1 : Gestion des secrets avec HashiCorp Vault

Problème : Les mots de passe sont stockés en clair dans le fichier `.env`, vulnérable en cas de compromission de l'hôte.

Solution : Remplacer le fichier `.env` par **HashiCorp Vault** ou **Docker Secrets** (en mode swarm). Les secrets seraient injectés dynamiquement dans les conteneurs au moment de leur création, sans jamais être stockés en clair sur le disque. Cela renforce la sécurité des credentials et permet une gestion fine des accès.

Recommandation n°2 : Durcissement des conteneurs

Problème : Les conteneurs tournent par défaut avec l'utilisateur root et des systèmes de fichiers modifiables, augmentant la surface d'attaque.

Solution : Appliquer les bonnes pratiques de durcissement suivantes :

- user: "1000:1000" : Exécuter les processus avec un utilisateur non-root.
- read_only: true : Monter le système de fichiers en lecture seule, avec des volumes dédiés pour les écritures (logs, données temporaires).
- security_opt : Appliquer des profils seccomp (filtrage des appels système) et AppArmor pour restreindre les capacités du conteneur.

Recommandation n°3 : Alerting actif et escalade automatique

Problème : Le SOC est actuellement passif ; les alertes doivent être consultées manuellement dans Wazuh, ce qui peut retarder la réponse.

Solution : Configurer Wazuh pour envoyer des **notifications automatiques** vers **Slack, Microsoft Teams, ou par email** selon le niveau de criticité. Par exemple, une alerte de niveau 10 (brute force) déclencherait un message Slack dans le canal de l'équipe SOC. Dans TheHive, des règles d'**escalade automatique** peuvent être créées pour augmenter la sévérité d'un cas si des alertes similaires sont reçues dans un court intervalle, accélérant ainsi la prise de décision.

7. CONCLUSION

Ce projet nous a permis de mettre en pratique l'ensemble des compétences liées à la conteneurisation et à la cybersécurité. Nous avons appris à orchestrer une stack complète avec Docker Compose, en respectant les bonnes pratiques de segmentation réseau, de persistance des données et de gestion des secrets. La mise en place des healthchecks a renforcé notre compréhension des dépendances entre services. Les phases d'attaque (scan Nmap, brute force SSH, injection SQL) nous ont familiarisés avec les mécanismes de détection de Suricata et Wazuh, ainsi qu'avec la réponse à incident via TheHive selon le cycle PICERL. Enfin, la rédaction de ce rapport technique nous a formés à la documentation professionnelle d'une infrastructure de sécurité.

ANNEXES

- **Annexe A** – docker-compose.yml
- **Annexe B** – .env
- **Annexe C** – .env.example
- **Annexe D** – .gitignore
- **Annexe E** – Commandes utilisées
- **Annexe F** – Logs bruts
- **Annexe G** – vm.max_map_count configuration